

Asian University for Women
Department of Computer Science

Course ID: CSCI4353
Course Title: Cyber Security & Ethical Hacking

Machine Learning Approaches in PDF Malware Detection: An Empirical Study

Group Number: 06

Sarah Tahsin (ID: 220083)
Novera Ahmed Nirbachita (ID: 220113)
Nazia Zarin Riti (ID: 220018)

Submitted To: Professor Akhtari Zameel

Machine Learning Approaches in PDF Malware Detection: An Empirical Study

1st Sarah Tahsin
ID: 220083

2nd Novera Ahmed Nirbachita
ID: 220113

3rd Nazia Zarin Riti
ID: 220018

I. ABSTRACT

Portable Document Format (PDF) files remain one of the most common vectors for exploitation, as signature-based antivirus systems are facing challenges detecting novel malwares. This study addresses that gap and investigates machine learning approaches as an alternative against signature-based tools. The proposed model proceeds in two stages including a well-structured literature synthesis of peer-reviewed studies, a comparative analysis of implemented techniques, findings and limitations across accuracy, robustness and explainability, followed by a validation analysis on a single benchmark dataset, the CIC-Evasive-PDFMal2022 dataset. The validation stage uses this dataset to train on seven benchmark classifiers: Random Forest, XGBoost, SVM, Naive Bayes, AdaBoost, KNN and ANN against a signature-based baseline, under three configurations, 5-fold cross-validation, K-means evasion testing and SHAP/LIME explainability analysis. XGBoost achieved the strongest and most stable performance, with 99.50% F1-scores across all features and folds retaining near perfect recall against evasive malicious samples, whereas the signature based model lagged substantially (79.74% F1-score) due to high false positive rates rather than missed detections. Explainability analysis highlighted Javascript and metadata size as the most stable cross-model indicators of malicious content. These outcomes determine that ensemble tree-based models offer superior, computationally efficient, and interpretable detection compared to signature-based approaches. These findings gives direct implications for practical, resource-constrained malware screening deployments.

Index Terms—Machine Learning, Evasive Malware, Explainability, Signature-Based, XGBoost, Random Forest

II. INTRODUCTION

Portable Document Format (PDF) files have become widely used document formats in the world due to their cross-platform flexibility, format compatibility, and ease of sharing across multimedia. These flexibilities have made PDF documents one of the most exploited attack vectors for malware delivery, as malicious code and exploits can be easily embedded within these legitimate documents. Traditional signature-based antivirus softwares are becoming increasingly inadequate against these threats, since they can only identify previously seen malware and are easily bypassed by the novel or unseen variants. To address these shortcomings, Machine Learning (ML) models have emerged as promising alternatives. These

models offer to generalize beyond signature-based detection by learning patterns from PDF structure, metadata, and behavioral features [1]. Most of the literature can be grouped into three methodologies, including classical models, explainability, and result variations. Classical ensemble methods such as Random Forest, XGBoost, and AdaBoost have achieved high detection accuracy on datasets such as CIC-Evasive-PDFMal2022 [1]–[3], and explainability techniques such as SHapley Additive eXplanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME) have been integrated to make model decisions interpretable to security analysts [4], [5]. Despite these advances, reported results vary considerably across studies in terms of accuracy, robustness to evasive samples, computational overhead, and the specific features deemed important, making it difficult to draw consistent conclusions about which approaches are genuinely superior or which perform reliably across differing conditions [6]–[8].

These gaps motivated this study to get a quantified analysis and evaluation of ML PDF malware detection models. The study focuses on seeking the model’s performance, robustness, computational efficiency, explainability and resistance to evasive attacks on benchmark dataset CIC-Evasive-PDFMal2022 guided by the following research questions:

RQ1: What machine learning and deep learning techniques have been used in recent literature for PDF malware detection?

RQ2: How do ML-based PDF malware detection techniques compare with traditional signature-based methods in terms of detection accuracy, computational performance, and resistance to evasive attacks?

RQ3: Using a single dataset, how do different machine learning models compare in terms of classification performance, and how consistently do these models rank the most influential features for PDF malware detection?

The key contribution of this study is as follows:

- A structured synthesis and comparative results on ML-based PDF detection categorized by methodology.
- An original empirical evaluation of seven ML models, including Random Forest, XGBoost, SVM, Naive Bayes, AdaBoost, KNN, and ANN, against an existing signature-based baseline on a benchmark dataset, the CIC-Evasive-PDFMal2022.
- A quantified analysis on evasive resistance that classifies typical evasive malicious PDFs and evaluates recall degradation for each model, reflecting solid evidence for model reliability under structured variation.

- A cross-model explainability analysis using SHAP and LIME on the two best-performing classifiers, establishing which features are consistently influential across models and which are model-specific, offering practical guidance for feature prioritization in real-world detection pipelines.
- A stability analysis via 5-fold cross-validation quantifying the reliability of each model's performance across data splits, distinguishing models whose reported accuracy is robust from those that are highly split-dependent.

In the study, the existing sections are organized as follows:

i) Section III discussed the related work on the domain; ii) Section IV provides the proposed methodology; iii) Section V discusses methodologies and their implementation; iv) Section VI highlights the results; v) Section VI has the discussion and interpretation of the findings; vi) Section VII provides limitation and future work, followed by vii) Conclusion.

III. RELATED WORK

ML has become one of the most widely used approaches for detecting malicious PDF files, which replaces the traditional signature-based methods that can struggle against malicious and evasive attacks. In this case, the existing works can be categorized based on their differences in methodologies, such as classical ensemble and tree-based models, explainable approaches, and fuzzy and rule-based classifiers. Most of the PDF malware research is based on classical ensemble and tree-based classifiers. The most common benchmark dataset is the CIC-Evasive-PDFMal2022 dataset due to its balanced and unbiased design [9].

Early work by M. Issakhani et al. addressed the limitations of the Contagio dataset by introducing the CIC-Evasive-PDFMal2022 benchmark through K-means-based evasive sample generation and proposed a two-level stacking framework combining SVM, Random Forest, AdaBoost, and Logistic Regression. Although the approach demonstrated strong classification performance, it lacked adversarial robustness evaluation and feature-level interpretability [9]. Building upon the same dataset, Q. Abu Al-Haija et al. proposed an optimizable AdaBoost Decision Tree with hyperparameter tuning through random search and achieved significant detection accuracy with microsecond-level prediction latency. However, the study did not investigate robustness against evasive attacks or provide explanations of the selected features [3].

Z. S. Borno et al. compared seven machine learning classifiers on the same dataset and reported superior performance from ensemble models, particularly XGBoost and Random Forest, further confirming the effectiveness of tree-based learning for PDF malware detection [1]. Similarly, Khan et al. compared Naïve Bayes, KNN, Random Forest, SVM, and A1DE using ZeroR/Ranker feature selection under both hold-out and cross-validation settings. Although KNN achieved the highest performance, the study did not investigate robustness against evasive attacks, omitted computational cost analysis, and provided no explanation for KNN's superior performance or validation on independent datasets [8].

Several studies have explored alternative learning paradigms. S. Mejjauli and S. Guizani introduced a Fuzzy Unordered Rule Induction Algorithm (FURIA), achieving significant accuracy result while improving decision interpretability compared with conventional classifiers [7]. M. Binsawad proposed a Logistic Model Tree (LMT) that combined decision trees with logistic regression to enhance robustness, outperforming several traditional ML models [6].

However, the latest research work focuses on combining predictive success with model interpretability. G. S. Hossain et al. developed a 15,958-sample dataset having three classes (benign, malicious, evasive). This paper benchmarked Random Forest against seven other classifiers. Random Forest was identified as the best-performing model and was used to rank feature importance, assemble feature subsets, and create a final feature set. Nonetheless, explanation stability is not tested against evasive samples, and explainability was derived from a single randomly selected tree rather than a complete ensemble [4].

M. Elattar et al. developed a gradient boosting-based framework integrated with SHAP explanations, achieving significant accuracy and perfect recall while providing interpretable feature importance for security analysts. Although the model demonstrated excellent performance, its computational complexity and dependence on a large feature space may limit deployment in lightweight environments [5].

Focusing on feature representation rather than classifier design, S. Y. Yerima et al. proposed an enhanced 35-feature representation by extending PDFMal with anomaly-based structural features intended to resist obfuscation. The improved feature set enhanced Random Forest performance on the same dataset. Nevertheless, several anomaly features depend on heuristic assumptions regarding benign document behavior and remain unvalidated against sophisticated adversarial manipulations, resulting in only marginal gains over already strong classifiers [2].

In addition to studying structural features traditionally used in analysis, C. Y. Liu et al. created RGB images based on the structure of malicious PDF objects by employing SimHash and then used a VGG19 convolutional neural network for their classification. Despite proving effective for detection and revealing similarities in the structure of malicious PDF files, the visualization method lacks interpretability and is vulnerable to advanced obfuscation tactics [10]. On the other hand, A. Rajamurugan et al. developed ILLE, a PDF malware detector combining NLP-based metadata feature extraction with ML classification. Additionally, an explainability module and reporting interface based on SHAP and LIME were integrated in this work. Cross-validation was included against SVM across training epochs; ILLE consistently exceeded the performance of SVM. Despite claiming robustness against obfuscation, this work does not provide evasive-sample testing, and does not compare against structural feature-based detectors like Random Forest and XGBoost [11].

From previous detection approaches that rely on static structural or content-based classifiers vulnerable to adversarial

manipulation, P. P. PM and P. Hemavathi proposed a PDF-GAN framework which combined a generator, discriminator, adversarial classifier, and surrogate classifier to produce PDF malware variants that retain harmful functionality while appearing benign by using tree-structured feature abstraction trained on 15,000 samples. However, the technique lacks comparisons with traditional classifiers, antivirus software, and explanations for the structural characteristics that make them successful [12]. In parallel, N. Iqbal et al. proposed SETPA, a rule-based structural evasion framework with eight injection techniques that were optimized using entropy-based feedback and tested on PDFRate v.2 and Hidost. Nevertheless, SETPA's efficacy against ensemble or explainability-driven detectors has not been tested, and its static rule-based design might not generalize to detectors trained with adversarial retraining because it was only tested against two structurally focused detectors [13].

In conclusion, the combination of comparative analysis against signature-based baseline with ML models, resistance to evasive attacks, and computational performance analysis along with interpretation has not been performed in the existing literature on the CIC-Evasive-PDFMal2022 dataset, motivating the need for a unified empirical evaluation that jointly assesses classification performance, evasion robustness, computational efficiency, and explainability.

IV. PROPOSED METHODOLOGY

A. Overview of the Proposed System

- Step 1: The data preprocessing module prepares the raw dataset before model training.
- Step 2: Feature ranking identifies the most informative features for PDF malware detection.
- Step 3: Optimal Feature Combination Selection gives ranked features based on the variety of feature combinations.
- Step 4: The Detection Engine evaluates two fundamentally different malware detection strategies: the Machine Learning Classification Engine and the Signature-Based Detection Model (Baseline).
- Step 5: Performance Evaluation: Both detection approaches are evaluated using 5-fold stratified cross-validation to ensure robust and unbiased performance estimation.
- Step 6: Explainability Module to improve transparency and interpretability: XAI techniques are applied to the best-performing machine learning models.
- Step 7: Interpreted Detection Outcome produces the malware detection result by classifying each PDF as Benign or Malicious.

All seven steps of the proposed workflow is visually represented in Fig. 1

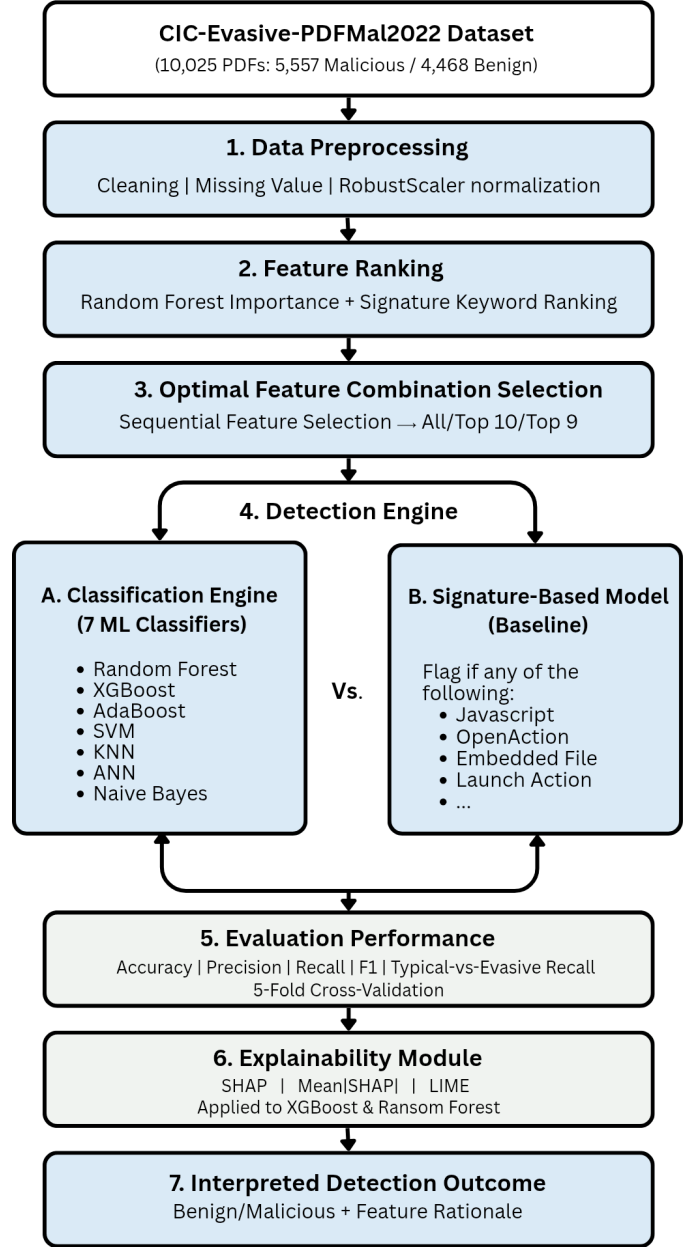


Fig. 1. Proposed System Workflow for PDF Malware Detection

B. Dataset Description

The study utilized the evasive PDF dataset CIC-Evasive-PDFMal2022 from the Canadian Institute for Cybersecurity [9]. The dataset consists of 10,025 records, with the distribution of 5,557 malicious records and 4,468 benign records. The visualization for class distribution of this dataset is included in Fig. 2, showing the existence of 55.4% Malicious and 44.6% Benign class PDFs. There are 31 features in the dataset, including metadata size, document encryption, size of the whole document, and number of pages.

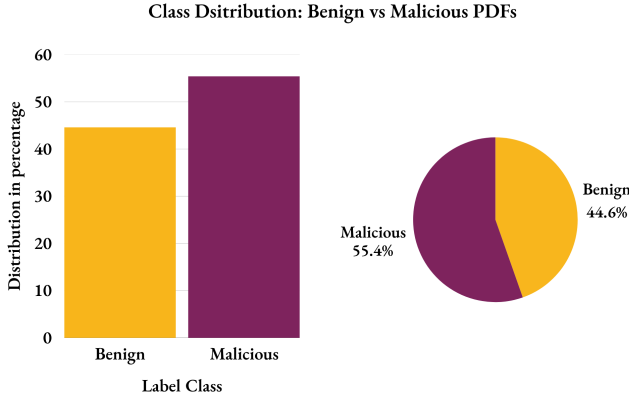


Fig. 2. Dataset Class Distribution

C. Preprocessing and Feature Selection

The data preprocessing module transforms the raw dataset into a standardized dataset for further experimentation. It includes cleaning, removing duplicate entries, handling missing values, and resolving inconsistencies in the datasets. To handle the missing value, a threshold value of 30% is used. Any column with a missing value above this threshold is removed as a data cleaning process. Any missing value was then handled with the column's mean value to keep the dataset numerically consistent, as the dataset has different numerical ranges and may contain extreme values. RobustScaler normalization is applied, making it less sensitive to outliers.

The study further processed feature selection through feature ranking and optimal feature selection. The feature selection is significant to focus on the most relevant feature while eliminating redundant or less relevant ones. At first, Random Forest feature importance is used to rank all extracted features according to their contribution in malware classification. The feature with the higher importance score gets greater influence in classifying PDF malware.

To identify the best selected combination of features for the model, the study used Sequential Forward Selection (SFS) by adding one feature at a time based on performance improvement. The process has been iterated until significant gain is achieved. According to the ranking result, the highest-performing feature combinations are all 31 features, the top 10 most important features are ranked as the 2nd most optimal combination, and the top 9 features are ranked as the 3rd most optimal feature combination.

This analysis demonstrates the accuracy of the early detection while re-identifying how consistently ML models rely on the same influential features and reducing computational cost.

V. METHODOLOGY & IMPLEMENTATION

A. Experimental Setup

All the experiments were conducted on a Google Colaboratory (Colab) CPU-only runtime with 2 virtual CPU cores (x86_64), 13.61 GB total RAM, and no GPU acceleration.

Python 3.12.13 was used with the following core libraries include Pandas 2.2.2, NumPy 1.0.1, scikit-learn 1.6.1, and XGBoost 3.3.0. Additionally, mlxtend (for sequential feature selection), shap and lime (for explainability), and matplotlib (for visualization) were used.

B. Research Procedure

The design of this research comprised two phases, which include, first, literature synthesis and, second, controlled empirical validation of the design on one benchmark test. In total, 13 peer-reviewed studies were detected and classified into three major methodological families and compared with respect to technology, key results, and reported limitations of the studies. This synthesis directly led to the choice of which model families and feature-ranking protocols would be forwarded to be empirically validated. The Fig. 3 represents the frequency of the models used across the previously mentioned papers, which is directly relevant to RQ1.

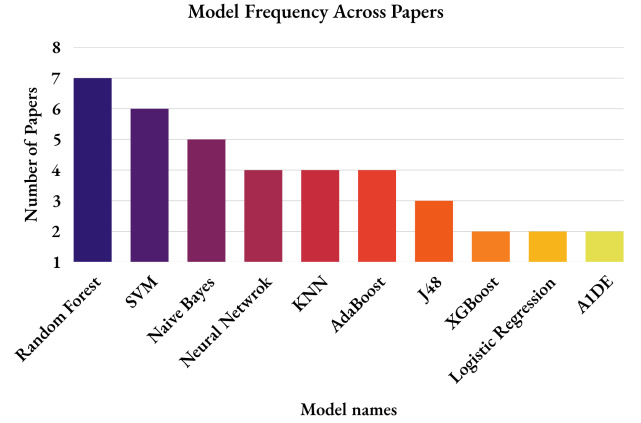


Fig. 3. Model Frequency Across Papers

In the empirical validation, data gathering, data cleaning, and missing values are performed first. This work used a 30% missing-value threshold to tackle numeric inconsistencies. Data normalization and scaling were performed using RobustScaler. Two different feature ranking procedures were performed: ranking by the Random Forest Gini importance on the entire feature set and ranking by individual recall for only the ten signature-style features, which are the types that are traditionally considered by signature-based detection. To find the optimal subset of the features, Sequential Forward Feature Selection (SFS) with a Random Forest base estimator was used to evaluate combinations of 1-15 features using 3-fold cross-validation F1-scoring, which resulted in the best-performing 14-feature combination, and top-10 and top-9 “compact” subsets were hand-picked for comparison with the full 31-feature set.

After data preprocessing and feature selection, seven ML/DL-based classifiers, including Random Forest, XGBoost,

SVM, Naive Bayes, AdaBoost, KNN, and ANN, plus a rule-based signature-based baseline, were trained and evaluated under three feature configurations, that includes (i) all 31 features, (ii) the top-ranked 10-feature combination, and (iii) the second-best 9-feature combination using an identical 80/20 stratified train/test split (random_state = 42) across all three, so results are directly comparable. On the other hand, signature-based baselines use the features it perceives as important.

Moreover, typical malicious and evasive malicious PDFs were separated from the malicious class by using unsupervised K-means clustering in order to analyze the malicious PDFs for evasion resistance. The resistance to evasive samples was then quantified for each subgroup for all eight detectors by measuring recall. Additionally, other local explanations (SHAP and LIME) were computed for the two best models (XGBoost and Random Forest, both on a subset of 300 samples for computational feasibility) to enable a direct comparison of feature-importance rankings between models. Lastly, stability validation was performed by running 5-fold stratified cross-validation for each of the seven ML models with the full feature set, with the results in terms of accuracy and F1-score mean of the 5-fold and the standard deviation across folds computed to evaluate and validate the consistency of the results across folds and the stability of the models in general.

C. Evaluation Metrics

To assess the model performance and address the research questions of the study, we used the following standard classification metrics, as well as problem-specific metrics. Let TP, TN, FP and FN be true positive (malicious PDFs correctly classified as malicious), true negative (benign PDFs correctly classified as benign), false positive (benign PDFs incorrectly classified as malicious), and false negative (malicious PDFs incorrectly classified as benign), respectively.

Accuracy is overall correctness in classification, which is reported throughout for RQ2 and RQ3, and is computed as:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (1)$$

Precision is the percentage of maliciously flagged PDFs that are actually malicious, defined as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

Recall is the percentage of genuinely maliciously created PDFs that are identified:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (3)$$

For the evasion-resistance analysis in RQ2, recall is considered the primary measure and is calculated separately for the K-Means-derived as typical malicious and evasive malicious. The harmonic mean of precision and recall is called the F1-score.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

In all of this study, the F1-score is used as the main model ranking criterion and in the Sequential Forward Feature

Selection, since it is more reliable than accuracy in case of class imbalance. Individual Recall is a supplemental feature-level metric being added to the ranking to account for the discriminative power of each of the 10 signature-style indicator features individually. For a given feature f greater than 0, it is defined as

$$\text{Individual Rec} = \frac{\text{Number of malicious test PDFs}}{\text{Total number of malicious test PDFs}} \quad (5)$$

This metric quantifies the degree to which each signature feature independently indicates malicious intent, similar to the reasoning behind traditional rule-based detection.

The cross-validated stability is measured using the average and standard deviation of the accuracy and F1-score over 5 stratified folds. The results from the low standard deviation will be quantitative support for the consistency aspect of RQ3, which states that a model's performance is not specific to a particular train/test split.

The Random Forest Gini importance and the SHAP mean absolute value per feature are two complementary feature importance rankings used to evaluate feature importance consistency. To address the feature-ranking component in RQ3, the top features are determined by the top-performing models identified by the two best-performing models in terms of model performance, that is, Random Forest and XGBoost.

VI. RESULTS

A. Feature Ranking

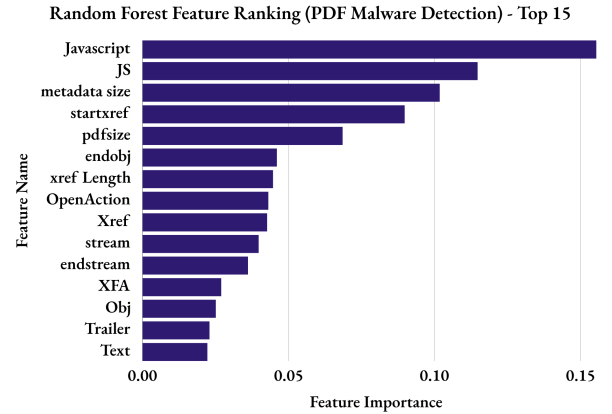


Fig. 4. Random Forest Gini Feature Importance Ranking (Top 15 features)

Fig. 4 and Fig. 5 represent the two artifacts generated by Step 2 of the pipeline in Fig. 1. The top 4 features that contribute most to the classification in the Random Forest model are Javascript, JS, metadata size, and startxref, as displayed on Fig. 4 which represents Gini-importance ranking across all 31 features. However, only the top 15 are included in the figure. The individual-recall ranking in the complementary signature style is shown in Fig. 5 and highlights again the highest-ranking features of the traditional rule-based ones, Javascript and JS, followed by a sharp drop to OpenAction.

Both of these numbers were directly from the output of Step 2 of the workflow, and they both indicate that the feature ranking step worked properly and produced non-degenerate, differentiated rankings, which is a prerequisite for the step that follows. This early agreement between the Random Forest Gini ranking and the individual-recall ranking points to both Javascript and JS as top features. This is the first sign of the ranking consistency RQ3 investigates.

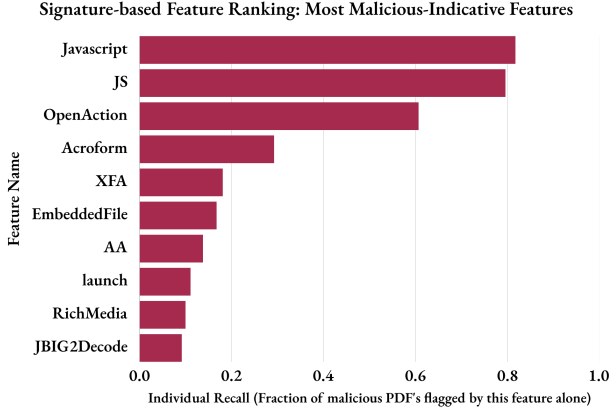


Fig. 5. Individual recall of each signature-based indicator feature

B. Preliminary Model Outputs

To demonstrate that the proposed pipeline was not only designed but fully executed, this section presents the empirical outputs generated at each stage of the implementation, together with an analysis of what each result indicates. Table I reports the performance of all seven ML classifiers on the complete 31-feature configuration, ranked by F1-score. Throughout the result comparative figures, the bold writing indicate best results while underlined values indicate second best results.

TABLE I
MODEL PERFORMANCE (ALL 31 FEATURES)

Model	Accuracy	Precision	Recall	F1-Score
XGBoost	99.45%	99.55%	99.46%	99.50%
<u>Random Forest</u>	99.10%	98.93%	99.46%	<u>99.20%</u>
ANN	98.95%	98.83%	99.28%	<u>99.06%</u>
KNN	97.81%	97.85%	98.20%	98.02%
AdaBoost	97.26%	97.91%	97.12%	97.51%
Signature-Based	74.02%	70.21%	92.56%	79.74%
SVM	78.25%	72.7%	97.3%	83.22%
Naive Bayes	57.51%	97.09%	24.03%	38.53%

XGBoost and Random Forest form a clear top tier, both exceeding 99% on every metric, whereas the signature-based baseline ranks seventh of eight, ahead only of Naive Bayes. It is worth noting that the signature-based method's weakness stems mainly from low precision (70.21%) rather than low recall (92.56%); in other words, its principal failure mode is the frequent misclassification of benign PDFs that legitimately make use of JavaScript or embedded objects, rather than an outright failure to detect malicious ones.

TABLE II
TOP 10 FEATURE & TOP 9 FEATURE F1-SCORE COMPARISON

Model	F1-Score (Top 9)	F1-Score (Top 10)
XGBoost	99.19%	99.24%
Random Forest	99.06%	99.19%
ANN	97.57%	98.2%
KNN	97.49%	97.35%
AdaBoost	96.86%	96.40%
Signature-Based	79.74%	79.74%
SVM	76.94%	77.20%
Naive Bayes	12.87%	12.55%

Across all the feature configurations tables I, andII XGBoost and Random Forest reveal a consistent performance hierarchy, achieving F1-scores of 99.50% and 99.20% on full feature set. ANN, KNN and Adaboost also exhibit only minor performance changes, indicating that most of the dataset's discriminative information is concentrated within a small subset of highly informative features. This observation is further supported by the 5-fold cross-validation results in Table V. Moreover, in this study the F1-score for XGBoost and Random Forest sustained even after reducing the feature from 31 to 9. Among all the models, Naive Bayes' F1-score decreased the most after the feature reduction (38.53% to 12.87%). However, as per Table tab:f1 comparison, the Naive Bayes F1-score increased when the feature dropped to 9 from 10.

C. Computational Performance Comparison

In order to study the computational performance aspect of RQ2, the training time, and inference latency of all seven ML classifiers, along with the signature-based model, were evaluated in the 31-feature setting showed in table III. This analysis, which complements the analysis of prediction performance, aims at determining whether a high-accuracy model is also computationally efficient enough for practical purposes.

TABLE III
COMPUTATIONAL PERFORMANCE

Model	Train Time	Inference-Full set	Inference/PDF
Signature-Based	0.000 s	0.002 s	0.001 ms
KNN	0.00295 s	0.109 s	0.055 ms
<u>Naive Bayes</u>	<u>0.00596 s</u>	0.002 s	0.001 ms
XGBoost	0.2643 s	0.007 s	0.004 ms
AdaBoost	0.4757 s	0.019 s	0.010 ms
Random Forest	2.0233 s	0.101 s	0.050 ms
ANN	3.932 s	<u>0.003 s</u>	<u>0.002 ms</u>
SVM	4.214 s	1.149 s	0.573 ms

Among all of the models, XGBoost is able to offer the optimal trade-off between detection performance and computational efficiency. It needs only 0.2643s to train the model and only 0.00369ms to scan each single PDF file, resulting in a throughput of roughly 270,906 PDFs/s. In combination with its F1-score of 99.50%, it shows that high detection performance can be obtained without sacrificing inference time. Random Forest also reaches a 99.20% F1-score with a much higher computational price. In contrast, the ANN classifier has a longer training time (3.932s) because of backpropagation. However, it produces very fast prediction results (0.00172ms

per PDF), having the second-fastest inference among the ML classifiers. This implies that recent ensembles and neural networks can at once produce highly accurate predictions and remain computationally efficient enough to detect malware.

Further, models demonstrate different trade-offs in terms of computations. For instance, AdaBoost has relatively reasonable training and inference times together with decent detection accuracy, while KNN has extremely fast training (0.003s) as it saves the data from training in memory. However, the prediction takes time (0.055 ms per PDF), since it involves calculation of the distance between the query PDF and all training samples available. SVM stands out as being the most computationally expensive classifier, which took 4.214s for training and 0.573ms per PDF for prediction, thus making it the slowest among all other classifiers used. This indicates that computational efficiency is not determined only by training time but also by the prediction method used by each of the learning algorithms.

The signature-based baseline approach is computationally the most efficient since there is no need for any model training, and it performs simple rule-based comparisons over predefined indicator features, which enables processing of about 1.24 million PDFs per second. Nevertheless, this computational efficiency is counterbalanced by poor prediction, where the model reaches a 79.74% F1-score and 70.21% precision. Consequently, even though the method of signatures is computationally efficient, it does not have predictive power, and hence it is inefficient compared to ML techniques.

D. Evasion-Resistance

The evasion-resistance analysis, in which recall was measured separately for the subgroups derived from K-means (typical malicious) and (evasive malicious), is reported in Table IV, providing a direct comparison of each model's ability to maintain detection performance under varying malicious PDF characteristics.

TABLE IV
RECALL ON TYPICAL VS. EVASIVE MALICIOUS PDFs

Model	Recall (Typical)	Recall (Evasive)	Recall Drop
Random Forest	100%	99.44%	0.56%
XGBoost	100%	99.44%	0.56%
ANN	100%	99.25%	0.75%
KNN	100%	98.13%	1.87%
SVM	100%	97.2%	2.80%
AdaBoost	100%	97.01%	2.99%
Signature-Based	97.5%	92.06%	5.44%
Naive Bayes	100%	21.2%	78.80%

All the seven ML achieved perfect recall on the smaller, more distinctly malicious cluster, whereas the signature-based baseline already missed close to three percent of even this comparatively straightforward subgroup, which suggests that its shortcomings are rooted in a more general precision-recall trade-off rather than an evasion-specific weakness as such. Random Forest and XGBoost sustained recall above 99% on the evasive subgroup, a drop of around half a percentage point from their performance on typical malware, while Naive

Bayes collapsed to a recall of 21.1%, failing to detect nearly 80% of evasive samples. Taken together, these results offer direct, quantified evidence bearing on the evasion-resistance component of RQ2.

E. Stability Validation

Table V represents the stability validation of all 31 features of the dataset on 7 ML models and the signature-based model. XGBoost and Random Forest both maintain standard deviations below 0.2% on either metric, confirming that their strong performance in Table I does not depend on a particularly favorable train/test split. Naive Bayes, by contrast, exhibits a standard deviation nearly two orders of magnitude larger, at $\pm 8.57\%$ accuracy and $\pm 15.81\%$ F1-score, indicating that its performance varies considerably depending on which portion of the data it is trained on. This disparity provides quantitative grounding for the consistency dimension of RQ3, distinguishing models whose reported performance is broadly reliable from those whose results are comparatively fold-dependent.

TABLE V
5-FOLD CROSS-VALIDATION STABILITY

Model	Accuracy	F1-Score
XGBoost	99.51% $\pm$ 0.12%	99.56% $\pm$ 0.11%
Random Forest	99.34% $\pm$ 0.08%	99.41% $\pm$ 0.07%
ANN	98.80% $\pm$ 0.19%	98.92% $\pm$ 0.17%
KNN	97.71% $\pm$ 0.13%	97.94% $\pm$ 0.12%
AdaBoost	97.63% $\pm$ 0.41%	97.85% $\pm$ 0.37%
SVM	77.96% $\pm$ 0.71%	83.02% $\pm$ 0.48%
Signature-Based	74.14% $\pm$ 0.90%	79.95% $\pm$ 0.65%
Naive Bayes	72.00% $\pm$ 8.57%	65.15% $\pm$ 15.81%

F. Explainability and Feature-Importance Consistency

SHAP values were used to examine the robustness of the two most successful models to converge on the same influential features, with both Random Forest and XGBoost models applied to the 300-instance test subsample.

Table VI is a side-by-side ranking of the top 20 features identified through SHAP for the Random Forest and XGBoost classifiers. The common factors identified by both models are Javascript and metadata size, implying that JavaScript code embedded within the PDF file and unusual metadata size are good model-independent detectors of a threat. Aside from these two factors, there is some difference in the order of importance assigned by the two algorithms to the other features such as, the Random Forest model attaches more importance to structural features, including JS, xref Length, and OpenAction, while XGBoost prefers startxref, endobj, and obj. Embedded files, header_version, and Colors received low discriminative power in both models, indicating that these three attributes have limited relevance when used in the algorithms under study. The similarity in both rankings indicates the reliability of the found significant predictors, and the inconsistency of rankings concerning medium-level attributes demonstrates the need for comparisons of explainability results obtained from various models based on a single model's interpretation.

Together with the F1 scores in Table I, II and the low fold-to-fold variance in Table V, this answers RQ3 directly: Random Forest and XGBoost are consistent both in performance (stable F1-score across feature sets and folds) and in feature ranking (SHAP/LIME agree on Javascript and metadata size as top predictors), with disagreement only on mid-tier features.

TABLE VI
SHAP FEATURE IMPORTANCE RANKING (RANDOM FOREST VS XGBOOST)

Rank	Random Forest Feature	XGBoost Feature
1	Javascript	Javascript
2	metadata size	metadata size
3	JS	startxref
4	startxref	JS
5	pdfsize	pdfsize
6	OpenAction	endobj
7	xref Length	OpenAction
8	xref	obj
9	endobj	text
10	text	xref Length
11	stream	XFA
12	endstream	pageno
13	XFA	images
14	obj	Colors
15	trailer	xref
16	pageno	stream
17	embedded files	embedded files
18	header_version	pages
19	Colors	header_version
20	images	Acroform

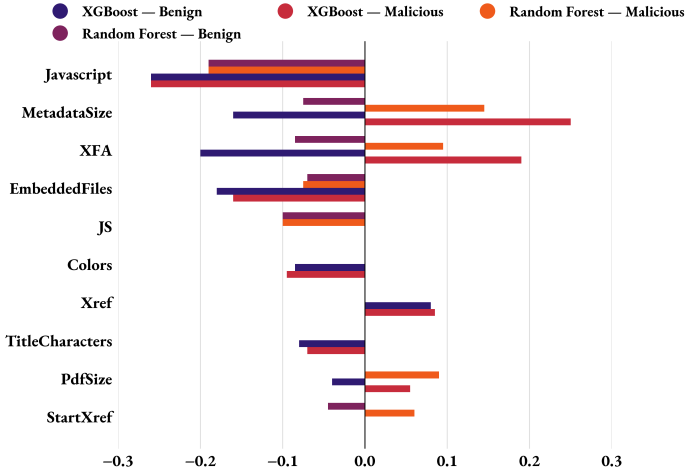


Fig. 6. LIME Feature Comparison - Top 10

Figure 6 shows the LIME weight contribution comparison between the XGBoost model and Random Forest model based on their predictions (benign and malicious classes). JavaScript is assigned a strongly negative weight for the benign class in both models, consistent with the SHAP finding that JavaScript is the dominant discriminative feature overall. The impact of metadata size on classification as malicious is equally significant in XGBoost, where it produces the largest positive weight in the entire comparison, consistent with the SHAP analysis. Overall, LIME analysis identifies metadata size and

XFA as consistent malicious-class indicators across both models, while JavaScript stands out as a consistent benign-class signal, together confirming the discriminative features already surfaced by SHAP.

G. Result Analysis

The results are benchmarked with the four best-performing same-dataset studies of [2], [3], [5], [9] since limiting to the same studies will rule out any impact due to the datasets themselves. This allows the identification of the specific methods that differentiate these studies from each other.

XGBoost’s 99.50% F1-score achieves almost equivalent results to all found baselines within the range of $\pm 0.75\%$, despite differences in protocol, which implies that performance on this dataset has approached saturation for classifiers from any family. A slightly better F1-score of 99.95% by M. Elattar et al. can be seen in the Table VII [5]. Relative to M. Issakhani et al., our work performed slightly better with an F1-score difference of 0.73% [9]. The table also shows our F1-score stands between Q. Al-Haija et al. and S. Y. Yerima et al. with not more than $\pm 0.7\%$ F1-score differences [2], [3].

TABLE VII
COMPARATIVE RESULT ANALYSIS

Study	Reported Best F1 score
Elattar et al. [5]	99.95%
Yerima et al. [2]	99.70%
Issakhani et al. [9]	98.77%
Abu Al-Haija [3]	98.80%
Our Study (XGBoost)	99.50%

VII. DISCUSSION

XGBoost and Random Forest outperformed other models since tree-based ensembles naturally capture non-linear feature interactions. Javascript combined with metadata size creates stronger signals than isolated features. Their hierarchical splitting handles PDF structural complexity effectively. Furthermore, the signature-based baseline’s low F1-score stemmed from high false positive rate as per (Table I), causing the Security Operations Centers to suffer from alert fatigue. Moreover, according to previously mentioned work in Table VII, our F1-score is slightly low, most likely due to absence of Synthetic Minority Over-sampling Technique (SMOTE) based class balancing which was done by M. Elattar et al. [5] to enhance the dataset. Parallely, S. Y. Yerima et al. [2] has implemented feature engineering for dataset enhancement. However, our work includes quantified evasion resistance and computational analysis, few contributions missing from most of the related works. Finally, as per Table III, XGBoost’s 0.004ms inference per PDF enables real-time scanning of approximately 270,000 PDFs per second, which is practical for email gateways. The 99.50% F1 score from Table I means approximately 1 false positive per 200 benign files, reasonable considering the expense of malware detection.

Our study answers all three research questions. For RQ1, literature synthesis identified Random Forest, SVM, and Naive Bayes as most frequently used classifiers. However, XGBoost

performed the best out of all seven classifiers. It outperformed the signature-based baseline by 19.76%, sustained 99.44% recall on evasive samples, and achieved 0.004ms inference per PDF, which confirms ML superiority in accuracy, evasion resistance, and speed, answering RQ2. For RQ3, XGBoost and Random Forest ranked top consistently, with SHAP/LIME independently confirming Javascript and metadata size as stable top features.

VIII. LIMITATIONS & FUTURE WORK

All the experiments were conducted on a 2-core, CPU-only Google Colab runtime. While it reflects a realistic resource-constrained environment, absolute training time, it may not generalize to production hardware. Additionally, the study was performed on a single benchmark dataset and all the results, reports, and evasive-resistance analysis are bounded by this one dataset, which causes a lack of generalized results. Furthermore, the typical-vs-evasive split is based on unsupervised K-means clustering on existing malicious samples, not adversarially crafted PDFs intended to fool a detector. This is a significantly weaker notion of evasion than the structural-injection attacks demonstrated in SETPA framework, which achieved 58–80% reductions in detection accuracy against structurally aware detectors like PDFRate v2.0 and Hidost through deliberate obfuscation (empty object streams, fake XREF entries, and benign metadata insertion) [13]. Due to CPU-only computational constraints, SHAP values were computed on a random smaller subsample, which lacks validation through repeated subsampling or bootstrap confidence intervals. On the other hand, Naive Bayes’s high fold-to-fold variance indicates that single-split results for weaker models should be interpreted cautiously.

However, the implementation of more advanced methodologies such as neural networks and the next generation of ML-based PDF detectors may gain higher accuracy and may be more suitable for predicting malicious attacks. Finally, more comprehensive work may increase reliability and resilience against growing evasive techniques.

IX. CONCLUSION

This study demonstrates that ensemble tree-based models, precisely XGBoost, outperform the other 7 ML classifiers, offering a superior performance against signature-based detection for PDF malware screening. This evaluation on the CIC-Evasive PDFMal2022 gained near perfect recall 99.46% with 99.50% F1-score and 99.55% precision, outperforming the signature-based model (79.74% F1). While SHAP and LIME explainability consistently identified Javascript and metadata size as the strongest features across models, its computational efficiency (0.264s training, 0.004ms inference per PDF) makes it adaptable in real setup. These findings provide a robust and comprehensive framework for deployability despite the limitations regarding the dataset.

REFERENCES

- [1] Z. S. Borno, N. Sakib, and S. S. Anwar, “Performance analysis of ensemble machine learning algorithms in pdf malware detection,” in *2023 IEEE 9th International Women in Engineering (WIE) Conference on Electrical and Computer Engineering (WIECON-ECE)*. IEEE, 2023, pp. 195–200.
- [2] S. Y. Yerima, A. Bashar, and G. Latif, “Malicious pdf detection based on machine learning with enhanced feature set,” in *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)*. IEEE, 2022, pp. 486–491.
- [3] Q. Abu Al-Haija, A. Odeh, and H. Qattous, “Pdf malware detection based on optimizable decision trees,” *Electronics*, vol. 11, no. 19, p. 3142, 2022.
- [4] G. S. Hossain, K. Deb, H. Janicke, and I. H. Sarker, “Pdf malware detection: toward machine learning modeling with explainability analysis,” *Ieee Access*, vol. 12, pp. 13 833–13 859, 2024.
- [5] M. Elattar, A. Younes, I. Gad, and I. Elkabani, “Explainable ai model for pdfmal detection based on gradient boosting model,” *Neural Computing and Applications*, vol. 36, no. 34, pp. 21 607–21 622, 2024.
- [6] M. Binsawad, “Enhancing pdf malware detection through logistic model trees,” *Computers, Materials, & Continua*, vol. 78, no. 3, p. 3645, 2024.
- [7] S. Mejjaoui and S. Guizani, “Pdf malware detection based on fuzzy unordered rule induction algorithm (furia),” *Applied Sciences*, vol. 13, no. 6, p. 3980, 2023.
- [8] B. Khan, M. Arshad, and S. Khan, “Comparative analysis of machine learning models for pdf malware detection: Evaluating different training and testing criteria,” *Journal of Cybersecurity*, vol. 5, p. 1, 2023.
- [9] M. Issakhani, P. Victor, A. Tekeoglu, and A. H. Lashkari, “Pdf malware detection based on stacking learning,” in *Icissp*, 2022, pp. 562–570.
- [10] C.-Y. Liu, M.-Y. Chiu, Q.-X. Huang, and H.-M. Sun, “Pdf malware detection using visualization and machine learning,” in *Data and Applications Security and Privacy XXXV*, K. Barker and K. Ghazinour, Eds. Cham: Springer International Publishing, 2021, pp. 209–220.
- [11] A. Rajamurugan, S. Desikan, J. DineshKumar, D. Sabarish, and C. Vasanathan, “Ille: Intelligent pdf detection mechanism based on natural language processing and machine learning techniques,” in *2025 Tenth International Conference on Science Technology Engineering and Mathematics (ICONSTEM)*, 2025, pp. 1–10.
- [12] P. P. PM and P. Hemavathi, “Pdf malware detection system based on machine learning algorithm,” in *2022 International Conference on Automation, Computing and Renewable Systems (ICACRS)*. IEEE, 2022, pp. 538–542.
- [13] N. Iqbal, F. B. Hussain, H. J. Hadi, N. Ahmad, and A. Shoker, “Setpa: Structural evasion techniques for pdf malware detection systems,” *Computers & Security*, p. 104775, 2025.